

AWS DevOps Interview Preparation

20 Frequently Asked Questions | Structured Point-wise Answers

Q1. Introduce yourself and describe day-to-day activities.

- Start with: Name → Degree (B.Tech Aeronautical Engg, RTM Nagpur, 2025) → Transition to DevOps through a structured upskilling path.
- ~9 months of internship at Hisan Labs: worked across AWS (EKS, EC2, VPC, S3, IAM, RDS, ECR, CloudFront, Route 53, Lambda), Kubernetes, Helm, Kustomize, Terraform, Jenkins, GitHub Actions.
- Day-to-day: Writing and reviewing Terraform modules for infra provisioning; managing Kubernetes deployments via Helm/Kustomize; monitoring via Datadog, Prometheus, Grafana.
- CI/CD pipeline maintenance in Jenkins and GitHub Actions; security scanning via Trivy and SonarQube; incident response, log analysis via ELK Stack.
- Anchor with ApnaKart project: 10-microservice e-commerce platform on AWS EKS — covers real-world scenarios like IRSA/OIDC, dual registry strategy, RDS per-service isolation.
- Interview tip: Quantify impact — '77% reduction in deployment time', '30% MTTR reduction', '99.9% uptime maintained'.

Q2. Give 5 Linux commands you use on a day-to-day basis.

- `tail -f /var/log/app.log` — Real-time log monitoring; essential for debugging running services.
- `ps aux | grep` — Check running processes; useful to identify resource-heavy or stuck processes.
- `netstat -tulnp` or `ss -tulnp` — List open ports and which process owns them; daily use during deployments.
- `df -h` and `du -sh *` — Disk usage check; critical in containerized environments to avoid OOM/disk-full failures.
- `grep -r 'ERROR' /var/log/` — Search logs recursively for error patterns; faster than opening files manually.
- Bonus worth mentioning: `curl -I` for HTTP health checks; `top/htop` for live system resource view.

Q3. A Java app runs on port 8089. How do you check which app is using which port in Linux?

- Step 1 — Use `lsof`: `lsof -i :8089` → Lists the process (PID, name) currently holding port 8089.
- Step 2 — Use `ss` (modern replacement for `netstat`): `ss -tulnp | grep 8089`
- Step 3 — Use `netstat` (legacy, still common in older distros): `netstat -tulnp | grep 8089`
- Step 4 — Kill the blocking process if needed: `kill -9` or `kill (graceful)`.
- Step 5 — Verify port is free: `lsof -i :8089` → Should return empty.
- Real scenario tip: In Kubernetes, port conflicts typically arise in `hostPort` or `NodePort` settings — check with `kubectl describe pod`.

Q4. What are inodes in Linux?

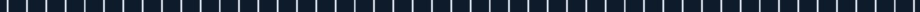
- An inode (index node) is a data structure in a Linux filesystem that stores metadata about a file — but NOT the filename or file data itself.
- Inode stores: file size, owner (UID/GID), permissions, timestamps (created/modified/accessed), number of hard links, pointer to data blocks on disk.
- Every file/directory has exactly one inode; identified by an inode number.
- Filename → inode mapping is stored in the directory entry, not the inode itself.
- Hard links: Two filenames pointing to the same inode. Deleting one does not delete the file until link count = 0.
- Soft links (symlinks): A separate inode pointing to a path, not the inode directly.
- Practical: `df -i` shows inode usage. A disk can run out of inodes even if space is available — common in systems with millions of small files.
- Check inode of a file: `ls -li filename` or `stat filename`

Q5. What are iptables?

- iptables is a Linux kernel-level firewall tool that controls incoming and outgoing network traffic based on rules.
- It operates by inspecting packets passing through the kernel's netfilter hooks.
- Key components — Tables: filter (default), nat, mangle, raw. Chains: INPUT, OUTPUT, FORWARD (in filter table).
- Rules are processed top-to-bottom; first match wins. Default policy applies if no rule matches.
- Common use: Block IP — `iptables -A INPUT -s 192.168.1.100 -j DROP`
- Allow port — `iptables -A INPUT -p tcp --dport 22 -j ACCEPT`
- NAT/Port forwarding — `iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080`
- In Kubernetes: kube-proxy uses iptables (or IPVS) to route Service traffic to Pod IPs — understanding this is critical for K8s networking interviews.
- Modern alternative: nftables (replaces iptables on newer systems like Ubuntu 20.04+).

Q6. Explain cron jobs. Also explain grep and sed commands with examples.

- CRON JOBS — A cron job is a scheduled task in Linux run by the cron daemon at defined time intervals.
- Crontab syntax: MIN HOUR DOM MON DOW COMMAND (e.g., 0 2 * * * /scripts/backup.sh → runs daily at 2 AM)
- Edit crontab: `crontab -e` | View: `crontab -l` | System-wide: `/etc/cron.d/`
- Use cases: Log rotation, DB backups, health check scripts, report generation, cleanup jobs.
- ██
- GREP — Searches for a pattern in files/output.
- Example: `grep 'ERROR' app.log` → finds all lines with 'ERROR'
- `grep -i 'error' app.log` → case-insensitive | `grep -r 'timeout' /var/log/` → recursive

- `grep -v 'DEBUG' app.log` → exclude lines with `DEBUG` | `grep -n 'WARN' app.log` → show line numbers
- 
- **SED** — Stream editor; find/replace or in-place file editing.
- Example: `sed 's/foo/bar/g' file.txt` → replace all 'foo' with 'bar' (output to stdout)
- `sed -i 's/old/new/g' config.yml` → in-place replacement (modifies the file directly)
- `sed -n '5,10p' file.txt` → print lines 5–10 only
- Real DevOps use: Update image tags in Helm values.yaml during CI pipeline.

Q7. Explain the flow of execution when you run a kubectl command. Which components are involved?

- 1. **kubectl CLI** — Reads `~/.kube/config` for cluster endpoint and credentials. Serializes request to JSON/YAML.
- 2. **Authentication & TLS** — **kubectl** authenticates to the API Server using client certificates, bearer token, or OIDC.
- 3. **API Server (kube-apiserver)** — The single entry point. Validates and authenticates the request, runs admission controllers (e.g., OPA Gatekeeper, ResourceQuota), then persists desired state to etcd.
- 4. **etcd** — Distributed key-value store. Stores all cluster state. API Server reads/writes here.
- 5. **Controller Manager (kube-controller-manager)** — Watches API Server for state changes. E.g., Deployment Controller detects desired replicas $\neq$ actual $\rightarrow$ creates ReplicaSet $\rightarrow$ creates Pods.
- 6. **Scheduler (kube-scheduler)** — Watches for unscheduled Pods. Scores nodes based on resource availability, affinity, taints/tolerations $\rightarrow$ assigns Pod to best node.
- 7. **kubelet (on the target node)** — Watches API Server for Pods assigned to its node. Calls the container runtime (containerd/CRI-O) via CRI to pull image and start container.
- 8. **kube-proxy** — Updates iptables/IPVS rules on each node to route Service traffic to Pod IPs.
- 9. **Response** — API Server returns the result (created/updated object) back to **kubectl**, which formats and prints it.
- Interview tip: For 'kubectl get pods' $\rightarrow$ flow stops at step 3 (read from etcd cache). For 'kubectl apply' $\rightarrow$ full flow including controller reconciliation loop.

Q8. How many types of Services are there in Kubernetes? When to use which?

- 1. ClusterIP (default) — Exposes service on a cluster-internal IP. Only reachable within the cluster. Use for: internal microservice communication (e.g., frontend → backend).
- 2. NodePort — Exposes service on a static port (30000–32767) on every node's IP. Accessible externally via NodeIP:NodePort. Use for: dev/testing when a load balancer is unavailable.
- 3. LoadBalancer — Provisions a cloud provider's external load balancer (AWS ELB, GCP LB). Best for production external traffic. Use for: exposing public-facing applications.
- 4. ExternalName — Maps a Service to an external DNS name (e.g., db.external.com). No proxying; just DNS CNAME aliasing. Use for: connecting to external services by a stable in-cluster DNS name.

- 5. Headless Service (ClusterIP: None) — No virtual IP assigned; returns Pod IPs directly via DNS. Use for: StatefulSets (CockroachDB, Kafka) where direct Pod-to-Pod addressing is needed.
- Interview tip: In your ApnaKart project — microservices use ClusterIP internally; the API Gateway uses LoadBalancer for external traffic; databases use Headless Services.

Q9. What does Ingress do in Kubernetes?

- Ingress is an API object that manages external HTTP/HTTPS traffic routing to internal Services — acting as a Layer 7 load balancer / reverse proxy.
- It requires an Ingress Controller (e.g., NGINX Ingress, AWS ALB Ingress Controller, Traefik) to be installed in the cluster — the Ingress resource alone does nothing.
- Capabilities: Host-based routing (api.myapp.com → api-service), Path-based routing (/checkout → checkout-service), TLS termination (HTTPS with cert-manager), Rate limiting, Auth middleware.
- Vs. LoadBalancer Service: One Ingress controller can handle routing for many services — more cost-efficient than one LB per service.
- Example rule: host: app.com, path: /api → backend: api-service:8080
- In ApnaKart: NGINX Ingress Controller used; cert-manager handles TLS; path-based routing separates frontend and backend traffic.

Q10. How to stop pod-to-pod communication within a namespace?

- Answer: Use Kubernetes NetworkPolicies to restrict traffic between Pods.
- By default, all Pods in a namespace can communicate with each other freely. NetworkPolicy changes this.
- Deny-all ingress policy for a namespace — apply a policy with podSelector: {} and no ingress rules: this blocks all inbound traffic to all pods in the namespace.
- Then create allow policies selectively: allow only specific Pods (by label) to communicate.
- Requires a CNI plugin that supports NetworkPolicy (Calico, Cilium, Weave Net). Flannel alone does NOT enforce NetworkPolicies.
- Example: Deny all intra-namespace traffic, then allow only frontend pods (app: frontend) to reach backend pods (app: backend) on port 8080.
- For zero-trust isolation: combine NetworkPolicy with ServiceAccount-based RBAC and mTLS via a service mesh (Istio/Linkerd).
- Interview trap: Saying 'just use namespaces to isolate' is wrong — namespaces alone do NOT restrict network traffic.

Q11. If given a task to deploy CockroachDB in Kubernetes, how would you do it?

- Step 1 — Use StatefulSet, not Deployment: CockroachDB requires stable network identities and persistent storage per Pod.
- Step 2 — Use Headless Service: Allows each Pod to be addressed individually (cockroachdb-0.cockroachdb, cockroachdb-1.cockroachdb) — required for cluster node discovery.

- Step 3 — PersistentVolumeClaims (PVC): Use volumeClaimTemplates in the StatefulSet to provision a dedicated PVC per Pod (e.g., 100Gi SSD via gp3 StorageClass).
- Step 4 — Init containers or cluster init job: Run cockroach init only once (on the first node) to bootstrap the cluster.
- Step 5 — Pod Disruption Budget (PDB): Set minAvailable: 2 to ensure quorum is maintained during voluntary disruptions.
- Step 6 — Anti-affinity rules: Spread Pods across different nodes/AZs to prevent a single node failure from losing quorum.
- Step 7 — Use the official Helm chart or CockroachDB Operator for production — handles init, rolling upgrades, and backup CronJobs.
- Step 8 — Expose a LoadBalancer or Ingress for the Admin UI (port 8080) and SQL port (26257) for application connections.
- Interview tip: Mention that ordered Pod startup (0 → 1 → 2) in StatefulSets is why CockroachDB works here — unlike Deployments which start Pods in arbitrary order.

Q12. What is VPC Peering vs Transit Gateway? If A peers with B and B peers with C, can A talk to C?

- VPC PEERING — A direct, private connection between two VPCs (same or cross-account, same or cross-region). Traffic does not traverse the public internet.
- Key limitation: NON-TRANSITIVE. If $A \leftrightarrow B$ and $B \leftrightarrow C$, A CANNOT talk to C through B. You must create an explicit $A \leftrightarrow C$ peering.
- Use case: Small number of VPCs (2–5); simple, low-latency direct connections; no central hub needed.
- 
- TRANSIT GATEWAY (TGW) — A centralized network hub that connects multiple VPCs and on-premises networks via a single gateway.
- TRANSITIVE by design: All VPCs attached to the same TGW can communicate with each other (subject to route tables).
- Supports: VPC-to-VPC, VPC-to-VPN, VPC-to-Direct Connect, cross-account, cross-region peering.
- Use case: Large-scale, hub-and-spoke network topology with 10+ VPCs; centralized egress/ingress; shared services VPC.
- DIRECT ANSWER: A peers with B, B peers with C → A CANNOT reach C via peering (non-transitive). Use Transit Gateway if transitive routing is required.
- Cost note: TGW charges per attachment hour + per GB data processed; peering has no hourly cost.

Q13. Difference between NACLs and Security Groups in AWS?

- SECURITY GROUPS — Stateful, instance-level (ENI-level) virtual firewall.
- Stateful: If inbound traffic is allowed, the response is automatically allowed regardless of outbound rules.
- Operates at: EC2 instance / ENI level. Applied when traffic reaches the instance.

- Rules: Allow rules only (no explicit deny). Evaluated as a whole — all rules are checked.
- Default: All inbound denied, all outbound allowed.
- 
- NACLs (Network ACLs) — Stateless, subnet-level firewall.
- Stateless: Must explicitly define both inbound AND outbound rules for a two-way connection.
- Operates at: Subnet boundary. Traffic is filtered before reaching any instance in the subnet.
- Rules: Both Allow and Deny rules. Rules evaluated in order by rule number (lowest first); first match wins.
- Default NACL: Allows all traffic. Custom NACL: Denies all traffic until rules are added.
- Use case: NACLs for broad subnet-level blocks (e.g., block an entire CIDR range); Security Groups for fine-grained per-instance control.
- Interview tip: NACLs are the first line of defense at the subnet; Security Groups are the second at the instance.

Q14. You added a DENY rule at priority 101 and an ALLOW rule at priority 95 for a public IP. What happens?

- Answer: The ALLOW rule (priority 95) wins and the traffic IS allowed.
- In Azure NSG and AWS NACL, rules are evaluated in ascending priority order — LOWER number = HIGHER priority.
- Priority 95 < Priority 101, so the ALLOW rule at 95 is evaluated FIRST.
- When the ALLOW rule at 95 matches, processing stops — the DENY at 101 is never reached.
- Result: Traffic from the resigned employee's IP is ALLOWED, not denied. This is a misconfiguration.
- Correct fix: Assign the DENY rule a lower priority number than the ALLOW rule (e.g., DENY at priority 90, ALLOW at 95).
- General rule: In priority-based firewalls (Azure NSG, AWS NACL), always assign DENY rules lower numbers (higher priority) than conflicting ALLOW rules.
- Real-world risk: This exact scenario causes security incidents — always audit rule ordering after adding deny rules for offboarded employees.

■ **Interview trap:** Many candidates assume higher number = higher priority. It is the opposite in both Azure NSG and AWS NACL.

Q15. Explain the types of Load Balancers in AWS Cloud.

- 1. Classic Load Balancer (CLB) — Legacy, Layer 4 & 7. Supports both TCP and HTTP. No host/path-based routing. Being phased out — avoid in new architectures.
- 2. Application Load Balancer (ALB) — Layer 7 (HTTP/HTTPS/WebSocket). Supports host-based and path-based routing, request redirection, sticky sessions, Lambda targets, WAF integration. Best for microservices and REST APIs.

- 3. Network Load Balancer (NLB) — Layer 4 (TCP/UDP/TLS). Handles millions of requests per second with ultra-low latency. Preserves client source IP. Best for real-time, high-throughput, non-HTTP traffic (gaming, VoIP, financial systems).
- 4. Gateway Load Balancer (GWLB) — Layer 3 (IP packets). Used to deploy, scale, and manage third-party virtual network appliances (firewalls, IDS/IPS, deep packet inspection). Routes traffic to inspection appliances transparently.
- Comparison summary: ALB for smart HTTP routing | NLB for raw performance/TCP | GWLB for security appliances | CLB for legacy only.

Q16. We are on a Zoom call right now. Which load balancer is being used?

- Answer: Network Load Balancer (NLB) — or equivalent Layer 4 load balancer.
- Zoom uses real-time audio/video streams over UDP (SRTP for media) and TCP/TLS for signaling — these are NOT HTTP/HTTPS requests.
- ALB works at Layer 7 (HTTP) and cannot efficiently handle persistent UDP media streams or low-latency real-time protocols.
- NLB operates at Layer 4 (TCP/UDP) with microsecond latency, no HTTP parsing overhead, and preserves the source IP — ideal for real-time communications.
- Zoom's infrastructure also uses its own media servers and a mix of TURN/STUN protocols — but at the cloud LB layer, NLB-equivalent is the appropriate choice.
- Broader rule: Any real-time media application (VoIP, video conferencing, gaming) → NLB. Web apps with routing logic → ALB.

Q17. Which load balancer acknowledges that it received the packets at the receiver end?

- Answer: Network Load Balancer (NLB) — because it operates at Layer 4 (TCP), and TCP itself provides acknowledgment (ACK) packets.
- TCP three-way handshake: SYN → SYN-ACK → ACK. The NLB (pass-through TCP mode) does not terminate the TCP connection; the backend server sends the TCP ACK directly to the client.
- In TCP Termination mode (NLB with TLS termination), the NLB ACKs the TCP connection itself.
- ALB (Layer 7) also uses TCP underneath, but by the time you're at HTTP layer, the transport-level acknowledgment is abstracted away.
- GWLB (Layer 3/IP packets) does NOT provide acknowledgment — it is connectionless at the IP level.
- CLB supports both Layer 4 TCP (with ACK) and Layer 7 HTTP.
- Key interview point: TCP ACK is a transport-layer mechanism. Load balancers that operate at Layer 4 (TCP) are the ones most directly involved in packet-level acknowledgment.

Q18. How to create and deploy a CI/CD pipeline across different environments with security in mind? What stages do you use?

- Pipeline design: Source → Build → Security Scan → Test → Artifact Publish → Deploy Dev → Deploy UAT → Deploy Prod

- Stage 1 — Source: Triggered on PR/push to feature, develop, or main branch (branch-to-environment strategy).
- Stage 2 — Build: Maven/Gradle build for Java; Docker image build. Tag image with Git SHA for traceability.
- Stage 3 — Security Scanning (SHIFT LEFT):
 - → SAST: SonarQube — static code analysis, code coverage, code smells.
 - → Container scanning: Trivy — scan Docker image for CVEs before pushing.
 - → Secret detection: GitLeaks / truffleHog — prevent secrets in code.
 - → Dependency check: OWASP Dependency-Check for vulnerable libraries.
- Stage 4 — Unit & Integration Tests: Fail fast — don't push if tests fail.
- Stage 5 — Artifact Publish: Push image to DockerHub (dev/test) or ECR (UAT/Prod). Sign image with Cosign for supply chain security.
- Stage 6 — Deploy to Dev: Auto-deploy on every merge to develop. Use Kustomize overlays or Helm --values dev-values.yaml.
- Stage 7 — Deploy to UAT: Auto-deploy on merge to release branch. Smoke tests post-deploy. Notify Slack on success/failure.
- Stage 8 — Deploy to Prod: Manual approval gate (Jenkins input step or GitHub Environment protection rules). Blue-green or canary rollout. Rollback trigger on Datadog alert breach.
- Secrets management: AWS Secrets Manager via External Secrets Operator — never hardcode secrets in pipeline or Helm values.
- In ApnaKart: Jenkins (Jenkinsfile with multi-branch pipeline) + GitHub Actions for PR validation. Branch dev → Dev env, release/* → UAT, main → Prod with approval.

Q19. Do you know how to monitor memory in EC2 instances? If yes, how?

- Key point: AWS CloudWatch does NOT collect memory metrics by default. Memory is an OS-level metric, not a hypervisor-level metric.
- Method 1 — CloudWatch Agent: Install the Amazon CloudWatch Agent on the EC2 instance. Configure `/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json` to collect `mem_used_percent`, `mem_available`, `swap` metrics. Metrics appear in CloudWatch under CWAgent namespace.
- Method 2 — Datadog Agent: Install Datadog Agent on EC2. It auto-collects memory (`system.mem.used`, `system.mem.pct_usable`) and sends to Datadog for dashboards and alerting. Used in ApnaKart for unified infra + APM monitoring.
- Method 3 — Prometheus + Node Exporter: Deploy `node_exporter` on EC2 → scrape with Prometheus → visualize in Grafana. Standard in Kubernetes environments (also works on bare EC2).
- Method 4 — Custom CloudWatch metric (Bash/Python script): Push custom metric via `aws cloudwatch put-metric-data` using a cron job that reads `/proc/meminfo`.
- Best practice in EKS: Use `kube-state-metrics` + `metrics-server` for Pod/node memory. Node Exporter DaemonSet for OS-level metrics. Datadog or Prometheus for alerting on memory thresholds.
- Interview tip: Always mention that CloudWatch's default 5 metrics (CPU, disk I/O, network, disk space) do NOT include memory — this is a common gotcha question.

Q20. Do you have experience in Python? Where and why have you used it?

- Yes — Python used across 3 areas: automation scripts, monitoring, and infrastructure tooling.
- 1. Infrastructure Automation: Boto3 (AWS SDK for Python) — wrote scripts to automate EC2 snapshot cleanup, S3 lifecycle policy enforcement, unused EIP identification, and IAM user audit reports.
- 2. CI/CD Utilities: Python scripts in Jenkins pipelines to parse Trivy JSON scan output, extract CVE counts, and fail the build if critical CVEs exceed threshold.
- 3. Monitoring & Alerting: Custom CloudWatch metric publisher using boto3 + psutil to push memory/disk metrics from EC2 instances not covered by default CloudWatch.
- 4. Kubernetes Tooling: Python scripts to parse kubectl JSON output (`kubectl get pods -o json`), identify pods in CrashLoopBackOff, and auto-trigger restart with notify-to-Slack.
- 5. Lambda Functions: Wrote Python-based Lambda functions for S3 event-triggered image processing and DynamoDB TTL cleanup in ApnaKart.
- Why Python over Bash for complex tasks: Better error handling, native JSON parsing, extensive libraries (boto3, requests, yaml), easier unit testing with pytest.
- Interview tip: Mention a specific script. E.g., 'I wrote a Python script using boto3 that scanned all ECR repositories, identified images older than 30 days, and deleted them — saving ~40% ECR storage costs.'